PROGRAMMING LANGUAGES

JavaScript

Modern ES2020+ JavaScript from types and closures to the event loop, async/await, modules, and the DOM — the language behind React and Node.

javascript · es2020 · async-await · closures · event-loop · prototypes · dom

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What JavaScript Is & Why It Matters
2. Setup & Getting Started
3. Types, Values & Coercion
4. Functions, Scope & Closures
5. Objects, Prototypes & Classes
6. The this Keyword & Binding
7. The Event Loop, Promises & async/await
8. Modules
9. Arrays, Iteration & Functional Methods
10. DOM & Fetch — Browser Basics
11. Cheat-Sheet Reference
12. Best Practices & Common Pitfalls
13. Interview Questions & Answers
14. Further Learning

What JavaScript Is & Why It Matters

JavaScript is the single-threaded, dynamically typed, prototype-based language that runs the web browser, the Node.js server, build tools, and the runtime beneath React, Vue, and Angular. Learn it deeply once and you can write front end, back end, CLIs, and mobile apps.

JavaScript (standardised as **ECMAScript**) ships a new edition every June. ES2015 (ES6) was the watershed release — `let` / `const`, arrow functions, classes, modules, promises. Everything from ES2020 onward (optional chaining, nullish coalescing, `BigInt`, top-level `await`, logical assignment) is the modern baseline this guide targets.

UBIQUITOUS

Runs in every browser, on servers (Node/Deno/Bun), in edge functions, and embedded devices. The only language native to the browser.

SINGLE-THREADED + ASYNC

One call stack, but an *event loop* plus non-blocking I/O lets it handle thousands of concurrent operations.

MULTI-PARADIGM

First-class functions enable functional style; prototypes and classes enable OOP. You pick per problem.

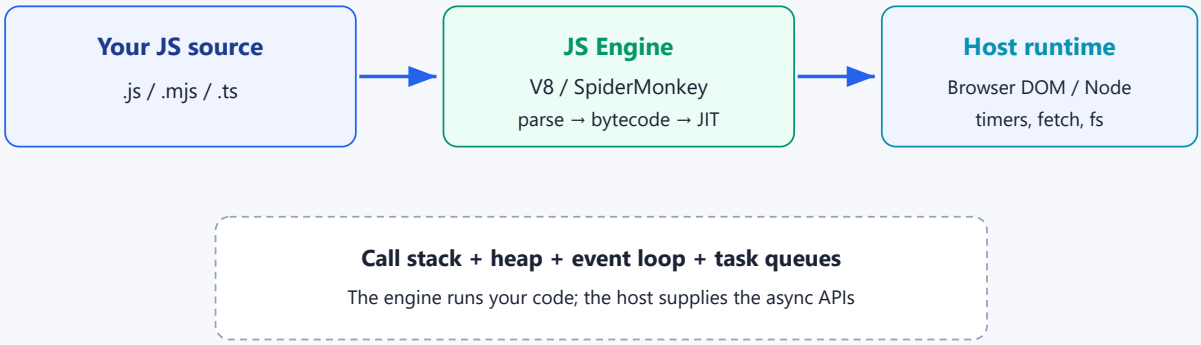


Fig 1 — JS source is executed by an engine (V8 etc.) embedded in a host runtime that provides I/O APIs.

Setup & Getting Started

You need almost nothing to start: open a browser console (`F12`). For real projects install Node.js (bundles `npm`).

BASH

```
# Install Node via the official installer or a version manager (nvm recommended)
node --version        # v22.x LTS as of 2026
npm --version

# Run a file
node app.js

# Start a REPL to experiment
node

# Initialise a project (creates package.json)
npm init -y

# Add a dependency and run a script
npm install lodash
npm run build
```

> **Tip**
> Use `"type": "module"` in `package.json` so `.js` files use ES module `import` / `export` syntax rather than legacy CommonJS `require`.

Types, Values & Coercion

JavaScript has **7 primitive types** plus objects. Primitives are immutable and compared by value; objects are compared by reference.

Type	Example	`typeof` result	Notes
number	`42` , `3.14` , `NaN`	`"number"`	IEEE-754 double; one type for int & float
bigint	`9007199254740993n`	`"bigint"`	Arbitrary-precision integers
string	`"hi"` , `` `t${x}` ``	`"string"`	UTF-16; immutable
boolean	`true`	`"boolean"`	
undefined	`undefined`	`"undefined"`	Absence of value (default)
null	`null`	`"object"`	Historic bug; intentional empty
symbol	`Symbol("id")`	`"symbol"`	Unique keys
object	`{}` , `[]` , `fn`	`"object"` / `"function"`	Everything non-primitive

JAVASCRIPT

```javascript
// Prefer const; use let when reassigning; never use var (function-scoped, hoisted)
const PI = 3.14159;
let count = 0;
count += 1;

// Strict equality (===) does NOT coerce. Loose equality (==) does — avoid it.
0 == "0";         // true  (coerced)  ⚠
0 === "0";        // false (no coercion) ✓
null == undefined;   // true  (the one useful == case)
NaN === NaN;         // false — use Number.isNaN()

// Falsy values (everything else is truthy): false, 0, -0, 0n, "", null, undefined, Na
if (!"") console.log("empty string is falsy");

// Nullish coalescing (??) only falls through on null/undefined, not on 0 or ""
const port = userPort ?? 8080;     // 0 would be kept; || would wrongly replace it
const name = input?.user?.name ?? "anon";  // optional chaining + nullish default
```

⚠ **Watch out**

`typeof null === "object"` and `typeof NaN === "number"`. To test arrays use `Array.isArray(x)`, not `typeof`.

Functions, Scope & Closures

Functions are first-class values: assign them, pass them, return them. Scope is *lexical* — determined by where code is written, not where it is called.

JAVASCRIPT

```
// Three ways to make a function
function declared(a, b) { return a + b; }          // hoisted fully
const expr = function (a, b) { return a + b; };    // not hoisted
const arrow = (a, b) => a + b;                      // concise; no own `this`

// Default, rest, and spread
function greet(name = "world", ...others) {
  return `hi ${name}` + (others.length ? ` and ${others.join(", ")}` : "");
}
const nums = [1, 2, 3];
Math.max(...nums);                 // spread into args → 3
const copy = [...nums, 4];         // spread into a new array
```

Closures

A **closure** is a function bundled with references to the variables of the scope where it was created. It keeps those variables alive after the outer function returns — the foundation of private state, memoisation, and callbacks.

JAVASCRIPT

```
function makeCounter() {
  let count = 0;                   // private — not reachable from outside
  return {
    inc() { return ++count; },     // closes over `count`
    get() { return count; },
  };
}
const c = makeCounter();
c.inc();  // 1
c.inc();  // 2
c.get();  // 2 — `count` persisted because the returned functions reference it
```

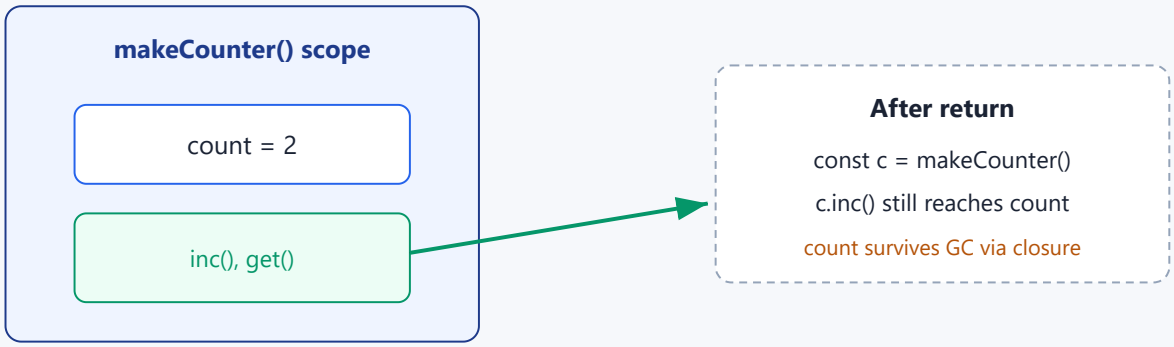


Fig 2 — The returned methods close over `count`, keeping that variable alive on the heap after the function returns.

Objects, Prototypes & Classes

JavaScript inheritance is **prototypal**: every object has a hidden link (`[[Prototype]]` , read via `Object.getPrototypeOf`) to another object. Property lookups walk this chain until found or `null` .

JAVASCRIPT

```
const animal = {
  describe() { return `${this.name} says ${this.sound}`; },
};
const dog = Object.create(animal);   // dog's prototype is `animal`
dog.name = "Rex";
dog.sound = "woof";
dog.describe();                      // "Rex says woof" — method found on prototype

// `class` is syntax sugar over prototypes
class Shape {
  #sides;                            // true private field (ES2022)
  constructor(sides) { this.#sides = sides; }
  get sides() { return this.#sides; }
  area() { return 0; }
  static unit() { return new Shape(1); }   // static = on the class, not instances
}
class Square extends Shape {
  constructor(side) { super(4); this.side = side; }
  area() { return this.side ** 2; }         // override
}
new Square(3).area();   // 9
```

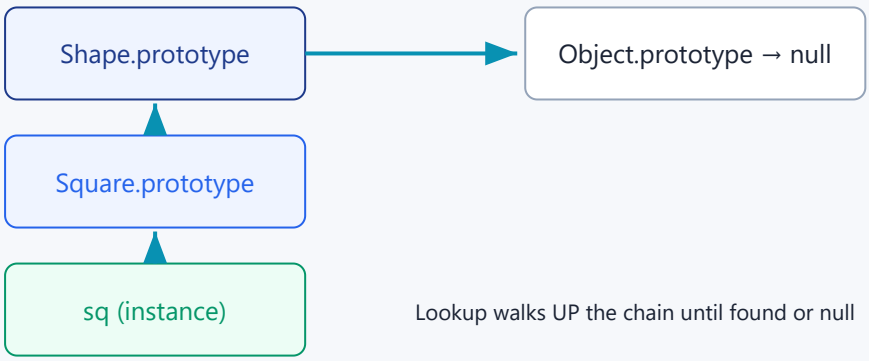


Fig 3 — The prototype chain. A missing property on `sq` is searched on each prototype in turn.

Destructuring & useful object APIs

JAVASCRIPT

```
const user = { id: 1, name: "Ada", role: "admin" };
const { name, role: title } = user;        // rename on extract → title === "admin"
const [first, , third] = [10, 20, 30];     // skip holes → first=10, third=30

Object.keys(user);     // ["id","name","role"]
Object.entries(user);  // [["id",1],["name","Ada"],["role","admin"]]
const clone = { ...user, role: "user" };   // shallow copy + override
Object.freeze(clone);  // shallow immutability
```

The `this` Keyword & Binding

`this` is decided by *how a function is called*, not where it is defined — except arrow functions, which capture `this` lexically.

Call form	What `this` is
`fn()` (plain)	`undefined` in strict mode (else global)
`obj.fn()`	`obj` (the receiver)
`new Fn()`	the freshly created instance
`fn.call(x)` / `fn.apply(x)`	`x` (explicit)
`fn.bind(x)`	permanently bound to `x`
arrow `() => ...`	inherited from the enclosing scope

JAVASCRIPT

```
const timer = {
  seconds: 0,
  startBroken() {
    setInterval(function () { this.seconds++; }, 1000); // ⚠ `this` is NOT timer
  },
  startFixed() {
    setInterval(() => { this.seconds++; }, 1000);       // ✓ arrow keeps `this`
  },
};
```

⚠ **Classic pitfall**

Passing a method as a callback (`element.addEventListener("click", obj.handle)`) loses `this` . Fix with `obj.handle.bind(obj)` or an arrow wrapper.

The Event Loop, Promises & async/await

JavaScript runs on one thread, so it never blocks on I/O. Instead, async work is handed to the host, and callbacks are queued to run once the call stack is empty — orchestrated by the **event loop**.

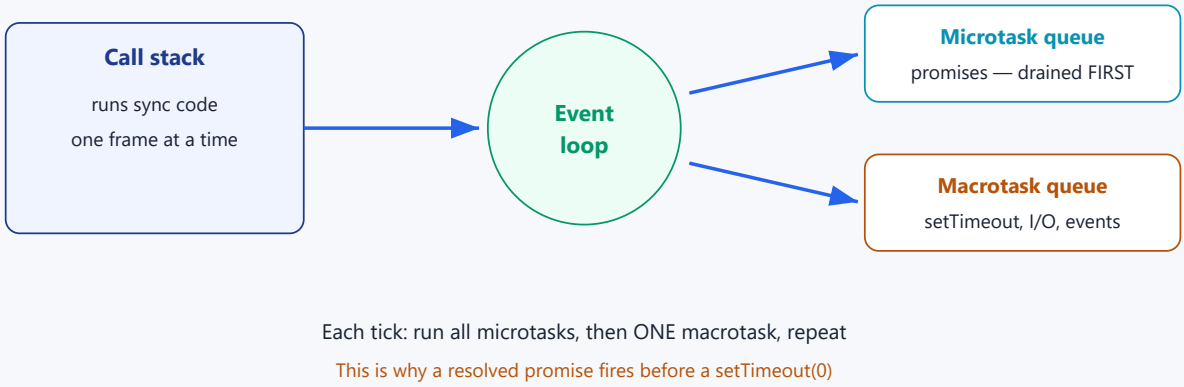


Fig 4 — The event loop empties the stack, drains all microtasks, then processes one macrotask per tick.

JAVASCRIPT

```javascript
console.log("1 sync");
setTimeout(() => console.log("4 macrotask"), 0);
Promise.resolve().then(() => console.log("3 microtask"));
console.log("2 sync");
// Output order: 1, 2, 3, 4  — microtasks beat macrotasks

// A Promise is a value that will exist later. States: pending → fulfilled | rejected.
function fetchUser(id) {
  return fetch(`/api/users/${id}`).then((res) => {
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return res.json();
  });
}

// async/await is syntactic sugar over promises — read it top to bottom
async function loadProfile(id) {
  try {
    const user = await fetchUser(id);              // pauses until resolved
    const [posts, friends] = await Promise.all([   // run concurrently, await both
      fetch(`/api/users/${id}/posts`).then((r) => r.json()),
      fetch(`/api/users/${id}/friends`).then((r) => r.json()),
    ]);
    return { user, posts, friends };
  } catch (err) {
    console.error("load failed:", err.message);
    throw err;                                      // rethrow or return a fallback
  }
}
```

Tip

Use `Promise.all` for "all must succeed", `Promise.allSettled` when you want every result regardless of failures, and `Promise.race` / `Promise.any` for the first to settle / first to succeed.

Modules

Modern code uses **ES modules** (ESM). Each file is its own scope; you explicitly export and import bindings. Node also supports legacy CommonJS (`require` / `module.exports`).

JAVASCRIPT

```javascript
// math.js — named exports + one default export
export const add = (a, b) => a + b;
export function mul(a, b) { return a * b; }
export default class Calculator { /* ... */ }

// app.js
import Calculator, { add, mul as multiply } from "./math.js";
import * as math from "./math.js";        // namespace import
const mod = await import("./math.js");    // dynamic import → lazy / code-splitting
```

	ES Modules	CommonJS
Syntax	`import` / `export`	`require` / `module.exports`
Loading	Static, async, tree-shakeable	Synchronous, dynamic
Where	Browsers & Node (default)	Legacy Node
Bindings	Live (read-only views)	Copied values

Arrays, Iteration & Functional Methods

Array methods are the workhorse of day-to-day JS. The functional trio `map` / `filter` / `reduce` replaces most hand-written loops.

JAVASCRIPT

```
const orders = [
  { id: 1, total: 30, paid: true },
  { id: 2, total: 70, paid: false },
  { id: 3, total: 50, paid: true },
];

orders.map((o) => o.total);                // [30, 70, 50] — transform
orders.filter((o) => o.paid);              // keep paid orders
orders.find((o) => o.id === 2);            // first match or undefined
orders.some((o) => !o.paid);               // true — any unpaid?
orders.every((o) => o.total > 0);          // true — all positive?

// reduce folds a list into one accumulated value
const paidRevenue = orders
  .filter((o) => o.paid)
  .reduce((sum, o) => sum + o.total, 0);   // 80

// Iterate with for…of (values) — prefer over for…in for arrays
for (const o of orders) console.log(o.id);

// Immutable-friendly helpers (ES2023): toSorted/toReversed/with don't mutate
const sorted = [3, 1, 2].toSorted();       // [1,2,3]; original untouched
```

Method	Returns	Mutates?	Use for
`map`	new array	no	1:1 transform
`filter`	new array	no	keep a subset
`reduce`	any value	no	aggregate/fold
`forEach`	`undefined`	no	side effects only
`flatMap`	new array	no	map + flatten one level
`sort`	same array	**yes**	in-place ordering
`push/splice`	varies	**yes**	in-place edits

DOM & Fetch — Browser Basics

In the browser, `document` exposes the DOM tree. You query nodes, mutate them, and listen for events.

JAVASCRIPT

```javascript
// Query
const btn = document.querySelector("#load");
const items = document.querySelectorAll(".item");   // static NodeList

// Create & insert
const li = document.createElement("li");
li.textContent = "New item";
li.classList.add("item");
document.querySelector("ul").append(li);

// Events — the handler receives an Event object
btn.addEventListener("click", async (event) => {
  event.preventDefault();
  const res = await fetch("/api/data", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ q: "hello" }),
  });
  if (!res.ok) throw new Error(res.statusText);
  const data = await res.json();
  console.log(data);
});
```

> **Note**
> `fetch` only rejects on network failure — an HTTP 404 or 500 still *resolves*. Always check `res.ok` yourself.

Mini worked example: a debounced search box

JAVASCRIPT

```
// Debounce: wait until the user stops typing before firing — a real-world closure use
function debounce(fn, delay = 300) {
  let timer;                                  // persisted via closure
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

const input = document.querySelector("#search");
const search = debounce(async (query) => {
  const res = await fetch(`/api/search?q=${encodeURIComponent(query)}`);
  const results = await res.json();
  render(results);
}, 400);

input.addEventListener("input", (e) => search(e.target.value));
```

Cheat-Sheet Reference

Task	Modern idiom
Default if null/undefined	`x ?? fallback`
Safe deep access	`a?.b?.c`
Clone array	`[...arr]` or `arr.slice()`
Shallow clone object	`{ ...obj }` or `structuredClone(obj)` (deep)
Merge objects	`{ ...a, ...b }`
Unique values	`[...new Set(arr)]`
Object → array	`Object.entries(obj)`
Array → object	`Object.fromEntries(pairs)`
Conditional assign	`obj.x ??= computeDefault()`
Format number	`n.toLocaleString()`
Parse/stringify JSON	`JSON.parse(s)` / `JSON.stringify(o, null, 2)`
Async delay	`await new Promise(r => setTimeout(r, ms))`



Best Practices & Common Pitfalls

DO

✓ **Good**

Default to `const`; use `===`; prefer pure functions and immutable updates; name async functions by what they return; always `await` or `return` a promise.

AVOID

✗ **Avoid**

`var`, loose `==`, mutating shared arrays, `for ... in` on arrays, floating promises (unhandled rejections), and blocking the main thread with heavy sync loops.

⚠ **Float math**

`0.1 + 0.2 === 0.3` is **false** (IEEE-754). Compare with an epsilon or work in integer cents. For money, consider `BigInt` or a decimal library.

JAVASCRIPT

```
// Pitfall: shared reference in a loop closure (var)
const fns = [];
for (var i = 0; i < 3; i++) fns.push(() => i);
fns.map((f) => f());   // [3, 3, 3] ⚠ — all share one `i`
// Fix: use let (new binding each iteration)
for (let j = 0; j < 3; j++) fns.push(() => j);   // [0, 1, 2] ✓

// Pitfall: forgetting await → floating promise, errors vanish
async function bad() { fetchUser(1); }   // ⚠ result + errors lost
async function good() { return await fetchUser(1); }
```

Interview Questions & Answers

What is the difference between `==` and `===`?

`===` compares type and value with no coercion. `==` coerces operands to a common type first, producing surprising results (`"" == 0` is true). Always use `===`; the one accepted `==` use is `x`

`== null` to catch both `null` and `undefined`.

Explain closures and give a real use.

A closure is a function together with the lexical environment it was created in, so it retains access to outer variables after the outer function returns. Uses: private state (module pattern), memoisation caches, debounce/throttle, and event handlers that remember configuration.

How does the event loop order `setTimeout(0)` vs a resolved promise?

The promise callback runs first. After each synchronous run completes, the loop drains the entire *microtask* queue (promise callbacks, `queueMicrotask`) before taking a single *macrotask* (timers, I/O). So microtasks always beat timers of the same tick.

What is `this` and how is it determined?

It is the call-time receiver, set by how the function is invoked: the object before the dot for method calls, the new instance for `new`, the explicit argument for `call` / `apply` / `bind`, and `undefined` (strict mode) for plain calls. Arrow functions ignore all of that and capture `this` lexically.

Explain prototypal inheritance.

Each object has an internal `[[Prototype]]` link to another object. Property reads that miss the object walk up this chain until found or `null`. `class` is sugar: methods live on `Class.prototype`, and `extends` links one prototype to another.

What is the difference between `null` and `undefined`?

`undefined` means a variable has not been assigned (the engine's default); `null` is an explicit "no value" you assign deliberately. `typeof undefined` is `"undefined"`; `typeof null` is `"object"` (a historic bug).

What does `const` actually guarantee?

That the binding cannot be reassigned — not that the value is immutable. A `const` object or array can still have its contents mutated. Use `Object.freeze` for shallow immutability.

How do you run async operations concurrently vs sequentially?

Sequentially: `await` each in turn. Concurrently: start them all (they return promises immediately) then `await Promise.all([ … ])`. Awaiting inside a loop serialises; mapping to promises then awaiting all parallelises.

What is hoisting?

Declarations are processed before execution. `function` declarations and `var` are hoisted (var initialised to `undefined`); `let` / `const` are hoisted but live in the "temporal dead zone" until their declaration line, so accessing them early throws.

Why is `0.1 + 0.2 !== 0.3`?

Numbers are 64-bit IEEE-754 doubles; 0.1 and 0.2 have no exact binary representation, so the sum is 0.30000000000000004. Compare with a tolerance (`Math.abs(a-b) < Number.EPSILON`) or use integer units.

Difference between `map` and `forEach`?

`map` returns a new transformed array and is for producing values; `forEach` returns `undefined` and is for side effects only. Chaining and functional pipelines use `map` ; you cannot `break` out of either (use `for ... of` or `some`).

What is event delegation?

Attaching one listener to a common ancestor and using `event.target` to handle events from many children, relying on bubbling. It is efficient for large/ dynamic lists and avoids per-element listeners.

Further Learning

- **MDN Web Docs** — the authoritative reference for every API and language feature.
- **"You Don't Know JS Yet"** (Kyle Simpson) — deep dives on scope, closures, `this` , and types.
- **TC39 proposals** — track upcoming ECMAScript features (temporal, records & tuples, decorators).
- **Node.js docs** — streams, the `fs` / `http` APIs, and the Node event loop phases.
- **TypeScript** — the natural next step: static types over the same language.
- **Build projects** — a debounced search UI, a small REST server, and a React app to apply closures, async, and modules in anger.